



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS ARARANGUÁ
CURSO DE ENGENHARIA DA COMPUTAÇÃO

Felipe Luan Schmitt

Trabalho 2: Contador Decrescente

Disciplina: Linguagem de Descrição de Hardware

LISTA DE FIGURAS

Figura 1 – Diagrama de blocos.	4
Figura 2 – Forma de ondas do divisor de clock.	8
Figura 3 – Forma de ondas do contador decrescente.	12
Figura 4 – Forma de ondas da Máquina de Estados Finitos.	16
Figura 5 – Forma de ondas do cronômetro decrescente.	22

SUMÁRIO

1	INTRODUÇÃO	3
2	DESENVOLVIMENTO	4
2.1	MÓDULOS INVIDIVIDUAIS	5
2.1.1	Divisor de clock	5
2.1.1.1	Implementação	5
2.1.1.2	<i>Testbench</i>	6
2.1.2	Contadores de minutos e segundos	8
2.1.2.1	Implementação	9
2.1.2.2	<i>Testbench</i>	10
2.1.3	Máquina de estados finitos	12
2.1.3.1	Implementação	13
2.1.3.2	<i>Testbench</i>	14
2.2	INTEGRAÇÃO DOS MÓDULOS	16
2.2.1	Implementação do cronômetro decrescente	16
2.2.2	<i>Testbench</i>	19
2.3	PROTOTIPAÇÃO NA PLACA NEXYS 1	22
2.3.1	Implementação do mapeamento	22
2.3.2	Validação prática	23
3	CONCLUSÃO	25

1 INTRODUÇÃO

Este documento apresenta o desenvolvimento e a implementação de um cronômetro decrescente em hardware, requisitado como o Trabalho T2 da disciplina de Linguagem de Descrição de Hardware. O objetivo principal do projeto consiste em descrever o circuito utilizando a linguagem VHDL, realizar a sua validação por meio de simulações com *testbenches* e, por fim, prototipar o sistema validado em uma placa de prototipação. O cronômetro projetado opera com precisão de um segundo, permitindo a configuração de um valor inicial e executando a contagem regressiva até que o tempo se esgote.

A placa de desenvolvimento alvo para a prototipação é a Digilent Nexys 1. Para a interface com o usuário, o projeto faz uso exclusivo dos periféricos nativos oferecidos pela placa, o que inclui chaves deslizantes para a inserção do tempo de carga, botões do tipo *push-button* para a emissão de comandos de controle, um diodo emissor de luz (LED) para sinalização de parada e quatro mostradores (*displays*) de sete segmentos multiplexados para a exibição visual dos minutos e segundos. O sincronismo do circuito lógico é garantido pela utilização da fonte de clock contida na própria placa, através de um cristal.

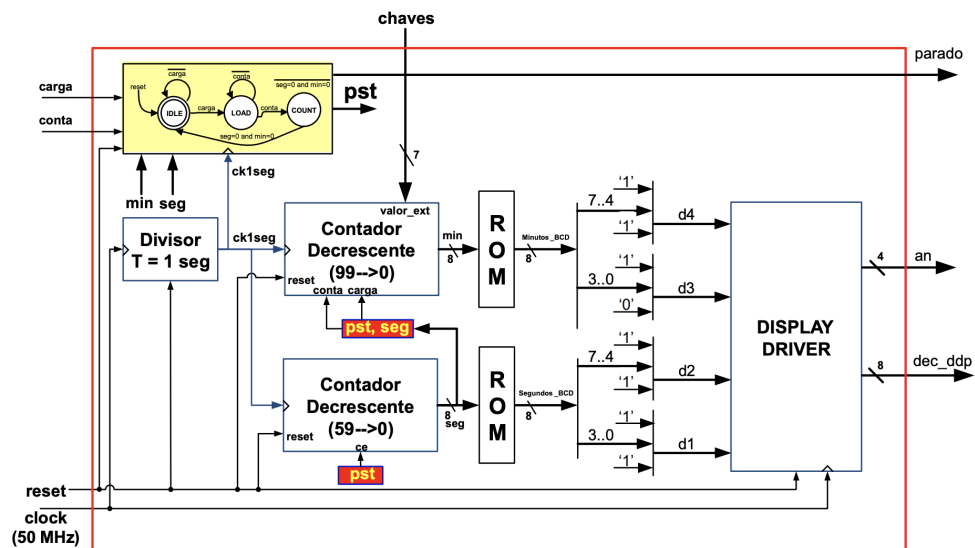
O desenvolvimento da arquitetura de hardware seguiu a metodologia de design hierárquico, particionando o escopo do problema em submódulos funcionais independentes. O controle do fluxo de operação do sistema é coordenado por uma Máquina de Estados Finitos (FSM, *Finite-State Machine*), encarregada de transitar o sistema entre os estados de repouso, carregamento de dados e contagem ativa. Cada bloco lógico gerado foi desenvolvido e testado isoladamente, garantindo que lógica de funcionamento esteja correta antes da integração completa no módulo de topo da hierarquia.

2 DESENVOLVIMENTO

Para o desenvolvimento do trabalho foi utilizado, conforme ensinado em sala de aula e recomendado pelo professor, um design hierárquico para assim modularizar as funcionalidades do cronômetro decrescente e facilitar nos testes e validações.

Os módulos foram baseados no diagrama de blocos proposto pelo professor apresentado abaixo, mas com algumas mudanças que serão abordadas ao longo do relatório.

Figura 1 – Diagrama de blocos.



Fonte: CALAZANS, N. L. V. Material didático da disciplina Linguagem de Descrição de Hardware.

O fluxo consiste no contador começar zerado e no estado IDLE, o usuário então pressiona o botão de carga, onde a máquina passara para o estado LOAD, ativando também o funcionamento das chaves (*switches*) da placa de prototipação para inserção do valor que será atribuído aos minutos na contagem, os segundos sempre começam em zero. Após definir o tempo do cronômetro, o usuário pressiona o botão de contagem, fazendo com que a FSM passe para o estado COUNT e inicie a contagem. A interrupção ocorrerá ao fim da contagem ou se o usuário pressionar o botão reset, voltando ao estado IDLE e zerando o contador.

Neste capítulo, primeiramente serão abordados os módulos de forma individual, com demonstrações do código, *testbenches* e formas de onda como demonstração da funcionalidade de cada um. Posteriormente, será demonstrada a integração de todos os módulos no arquivo topo da hierarquia, com validação utilizando o *testbench* fornecido pelo professor. Por fim, haverá a demonstração da prototipação na placa Nexys 1.

2.1 MÓDULOS INDIVIDUAIS

2.1.1 Divisor de clock

Para gerar os pulsos de clock do projeto foi utilizado o cristal presente na placa de prototipação. No caso da Nexys 1, é possível escolher entre 25 MHz, 50 MHz, e 100 MHz, para o presente trabalho foi utilizado 50 MHz.

O módulo divisor de clock é responsável por converter os 50 MHz em 1 segundo, pois se não o cronômetro iria contar todo o seu tempo em instantes. Para isso, o professor sugeriu dividir a frequência de clock por 25000000, formando uma onda quadrada perfeita e fazendo com que o clock passe meio segundo em '0' e meio segundo em '1', fechando 1 segundo. Entretanto, eu fiz de forma diferente, dividi os 50 MHz por 50000000, implementando um contador, que ao chegar ao limite, daria um pequeno pulso de enable para sinalizar que passou 1 segundo.

2.1.1.1 Implementação

Segue abaixo o código VHDL do divisor de clock.

divisor_clock.vhd

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY divisor_clock IS
6      GENERIC(
7          CLOCK_FREQ: integer := 50000000
8      );
9      PORT(
10         clock, reset: IN std_logic;
11         ck1seg: OUT std_logic
12     );
13 END ENTITY divisor_clock;
14
15 ARCHITECTURE behavior OF divisor_clock IS
16     SIGNAL contador: integer RANGE 0 TO CLOCK_FREQ := 0;
17 BEGIN
18     PROCESS(clock, reset)
19     BEGIN
20         IF reset = '1' THEN
21             contador <= 0;
22             ck1seg <= '0';
23         ELSIF rising_edge(clock) THEN
24             IF contador = (CLOCK_FREQ - 1) THEN
25                 contador <= 0;
26                 ck1seg <= '1';
27             ELSE
```

```

28         contador <= contador + 1;
29         ck1seg <= '0';
30     END IF;
31 END IF;
32 END PROCESS;
33 END behavior;

```

Fonte: Do autor (2026)

Dentro da entidade, criamos uma variável `GENERIC` chamada `CLOCK_FREQ`, atribuímos o valor de 50000000 para ela, que será o divisor do clock de 50 MHz, para, como dito antes, converter para 1 segundo. No `PORT`, definimos as entradas `clock` e `reset`, e como saída `ck1seg`.

Na `ARCHITECTURE`, criamos um sinal chamado `contador`, de tamanho igual ao `CLOCK_FREQ`, que será responsável por contar quantas subidas de clock ocorreram. No `PROCESS` temos como lista de sensibilidade o próprio clock e o reset, o que significa que ele será executado cada vez que um deles trocar de sinal.

A primeira condição do `PROCESS` é que se o reset estiver em '1', será atribuído '0' ao contador e à saída, resetando o divisor de clock. A segunda é a borda de subida do clock, dentro dela há outra condição se o contador chegou ao valor `FREQ_CLOCK - 1`, significando que passou exatamente 1 segundo e dando um pequeno pulso de enable no `ck1seg`. Se essa condição não se satisfaz, é incrementado 1 ao contador e o `ck1seg` permanece em '0'.

2.1.1.2 Testbench

Para comprovar o funcionamento do divisor de clock, foi desenvolvido o *testbench* apresentado a seguir.

divisor_clock_tb.vhd

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY tb_divisor_clock IS
5  END ENTITY tb_divisor_clock;
6
7  ARCHITECTURE behavior OF tb_divisor_clock IS
8      SIGNAL clock: std_logic := '0';
9      SIGNAL reset: std_logic := '0';
10     SIGNAL ck1seg: std_logic;
11 BEGIN
12     UUT: ENTITY work.divisor_clock
13         GENERIC MAP(
14             CLOCK_FREQ => 4
15         )
16     PORT MAP(

```

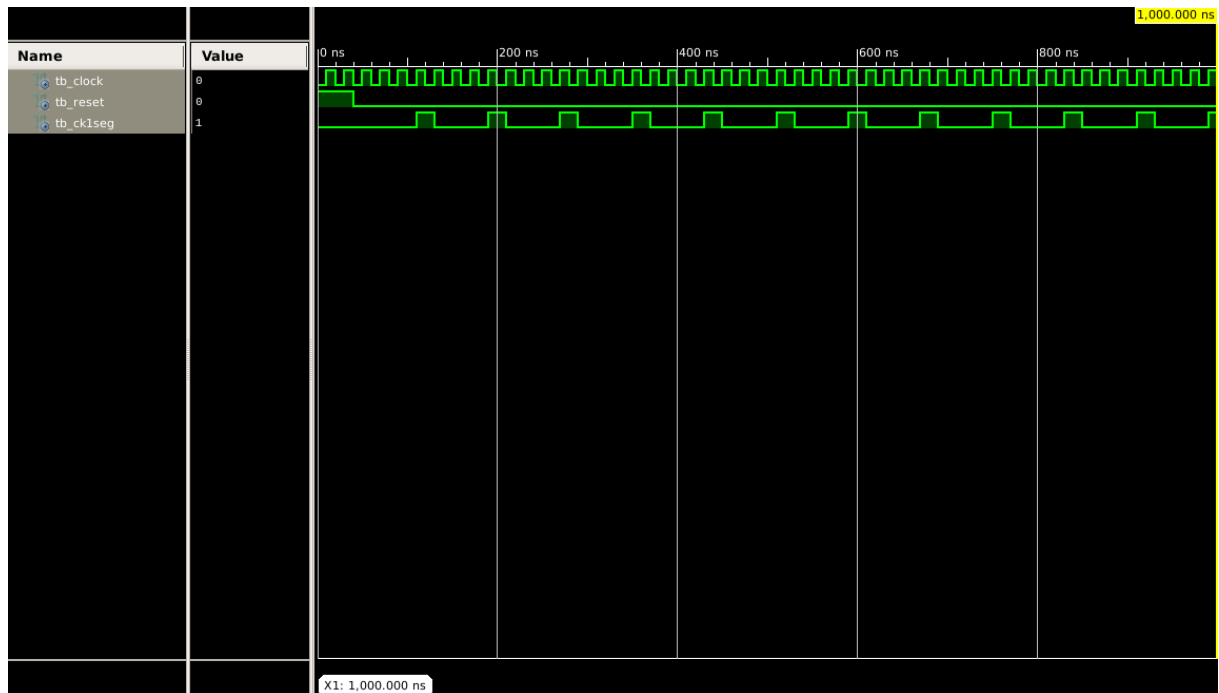
```
17         clock => clock ,
18         reset => reset ,
19         ck1seg => ck1seg
20     );
21     clock_process: PROCESS
22 BEGIN
23         clock <= '0';
24         WAIT FOR 10 ns;
25         clock <= '1';
26         WAIT FOR 10 ns;
27 END PROCESS;
28     reset_process: PROCESS
29 BEGIN
30         reset <= '1';
31         WAIT FOR 40 ns;
32         reset <= '0';
33         WAIT FOR 400 ns;
34         WAIT;
35 END PROCESS;
36 END behavior;
```

Fonte: Do autor (2026)

O código tem como função simular os pulsos de clock da placa Nexys 1 configurada a 50 MHz, sendo 20 ns cada ciclo, e dividir por uma variável genérica, comprovando o funcionamento do pulso de enable.

A simulação inicia com o reset em '1', garantindo que o contador interno zere, após 40 ns ele passa para '0' e o circuito passa a contar. Na imagem abaixo, que mostra as formas de onda do *testbench*, é possível visualizar o funcionamento do divisor, em que a cada 4 pulsos de clock, ocorre 1 pulso de enable.

Figura 2 – Forma de ondas do divisor de clock.



Fonte: Do autor (2026).

Também vale ressaltar que, como o reset é assíncrono, ele não depende do clock, então a qualquer instante que ele voltasse a ser '1', o contador travaria e voltaria a zero, até que o reset mude para '0' novamente.

2.1.2 Contadores de minutos e segundos

O objetivo deste módulo é responsável pelo registro contínuo dos minutos e segundos. A contagem opera dentro de restrições definidas nos requisitos do trabalho, em que os segundos vão de 0 à 59 e os minutos de 1 à 99, portanto, só é possível selecionar os minutos que serão contados, os segundos sempre começam em 0.

Para a contagem ocorrer, algumas condições precisam ser satisfeitas, começando pelo reset, que também é assíncrono, e precisa estar em '0' para que o fluxo funcione corretamente. Também é necessário o sinal da FSM dizendo que a máquina está no estado COUNT, com isso, a contagem irá ocorrer a cada subida de clock.

Entretanto, como o clock da placa opera em 50 MHz, foi adicionada a condição de que a contagem apenas decresce quando o pulso de `ck1seg` (vindo do divisor de frequência) estiver em nível alto ('1'). Isso garante que a subtração do tempo ocorra de forma estritamente controlada, exatamente uma vez por segundo.

Durante a especificação do projeto, o professor sugeriu uma abordagem utilizando dois submódulos de contagem distintos em cascata: um para os segundos e outro para os minutos, em que o módulo de segundos geraria um sinal de *carry out* para habilitar o decremento dos minutos. Contudo, optei por juntar ambos os módulos em um único bloco funcional. Essa decisão de

projeto simplifica o roteamento dos sinais de controle e facilita muito a operação de carga, que exige alterar os minutos e zerar os segundos simultaneamente. Além disso, garante que a virada do tempo (de 00 para 59) seja avaliada de forma mais prática e perfeitamente síncrona na mesma borda de clock.

2.1.2.1 Implementação

Segue abaixo o código VHDL do contador decrescente.

contador_dec.vhd

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY contador_dec IS
6      PORT(
7          clock, reset, carga, conta, ck1seg: IN std_logic;
8          chaves: IN std_logic_vector(6 DOWNTO 0);
9          seg_out: OUT std_logic_vector(7 DOWNTO 0);
10         min_out: OUT std_logic_vector(7 DOWNTO 0)
11     );
12 END ENTITY contador_dec;
13
14 ARCHITECTURE behavior OF contador_dec IS
15     SIGNAL cont_seg: integer RANGE 0 TO 59 := 0;
16     SIGNAL cont_min: integer RANGE 0 TO 99 := 0;
17 BEGIN
18     PROCESS(clock, reset)
19     BEGIN
20         IF reset = '1' THEN
21             cont_seg <= 0;
22             cont_min <= 0;
23         ELSIF rising_edge(clock) THEN
24             IF carga = '1' THEN
25                 cont_seg <= 0;
26                 IF to_integer(unsigned(chaves)) > 99 THEN
27                     cont_min <= 99;
28                 ELSE
29                     cont_min <= to_integer(unsigned(chaves));
30                 END IF;
31             ELSIF conta = '1' AND ck1seg = '1' THEN
32                 IF cont_seg = 0 THEN
33                     cont_seg <= 59;
34                     IF cont_min > 0 THEN
35                         cont_min <= cont_min - 1;
36                     END IF;
37                 ELSE

```

```

38         cont_seg <= cont_seg - 1;
39     END IF;
40 END IF;
41 END IF;
42 END PROCESS;
43 seg_out <= std_logic_vector(to_unsigned(cont_seg, 8));
44 min_out <= std_logic_vector(to_unsigned(cont_min, 8));
45 END behavior;

```

Fonte: Do autor (2026)

Na entidade, foram definidas todas as entradas de controle (`reset`, `carga`, `conta`), o referencial de tempo (`ck1seg`), a entrada de dados do usuário (`chaves`) e as saídas que irão para o mostrador (`seg_out` e `min_out`).

Dentro da `ARCHITECTURE`, foram criados dois sinais internos do tipo inteiro: `cont_seg` (limitado de 0 a 59) e `cont_min` (limitado de 0 a 99). O uso de inteiros com limites definidos simplifica as operações aritméticas de subtração e impede que o cronômetro assuma valores inválidos.

A lógica sequencial do `PROCESS` segue uma hierarquia estrita de prioridades. O `reset` assíncrono possui a maior prioridade. Na sua ausência, a cada subida de `clock`, o sistema verifica se o comando `carga` está ativo. Quando ativado, uma lógica de saturação avalia a entrada: se o valor convertido das chaves for superior a 99, o registrador de minutos é fixado em seu limite máximo de 99, caso contrário, o valor exato lido das chaves é salvo integralmente. Em ambos os cenários de carga, os segundos são forçados a zero. Por fim, se os comandos `conta` e `ck1seg` estiverem simultaneamente em '1', ocorre a subtração. A lógica verifica se os segundos chegaram a zero para realizar a "virada" para 59 e subtrair um minuto, possuindo ainda uma trava de segurança (`cont_min > 0`) para evitar que o tempo assuma valores negativos. Ao final, os valores inteiros são convertidos de volta para vetores lógicos de 8 *bits* para serem enviados às saídas.

2.1.2.2 Testbench

Para validar o funcionamento da contagem e, principalmente, a virada de um minuto para o outro, o seguinte ambiente de testes foi desenvolvido.

contador_dec_tb.vhd

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY tb_contador IS
5  END ENTITY tb_contador;
6
7  ARCHITECTURE behavior OF tb_contador IS

```

```
8     SIGNAL clock: std_logic := '0';
9     SIGNAL reset: std_logic := '0';
10    SIGNAL carga: std_logic := '0';
11    SIGNAL conta: std_logic := '0';
12    SIGNAL ck1seg: std_logic := '0';
13    SIGNAL chaves: std_logic_vector(6 DOWNTO 0) := (others => '0');
14    SIGNAL seg_out: std_logic_vector(7 DOWNTO 0);
15    SIGNAL min_out: std_logic_vector(7 DOWNTO 0);
16 BEGIN
17     UUT: ENTITY work.contador_dec
18         PORT MAP(
19             clock => clock,
20             reset => reset,
21             carga => carga,
22             conta => conta,
23             ck1seg => ck1seg,
24             chaves => chaves,
25             seg_out => seg_out,
26             min_out => min_out
27         );
28     clock_process: PROCESS
29 BEGIN
30         clock <= '0';
31         WAIT FOR 10 ns;
32         clock <= '1';
33         WAIT FOR 10 ns;
34     END PROCESS;
35     estimulos: PROCESS
36 BEGIN
37         reset <= '1';
38         WAIT FOR 40 ns;
39         reset <= '0';
40         WAIT FOR 40 ns;
41         chaves <= "0000010";
42         carga <= '1';
43         WAIT FOR 40 ns;
44         carga <= '0';
45         WAIT FOR 40 ns;
46         conta <= '1';
47         FOR i IN 1 TO 3 LOOP
48             WAIT FOR 60 ns;
49             ck1seg <= '1';
50             WAIT FOR 20 ns;
51             ck1seg <= '0';
52         END LOOP;
53         conta <= '0';
54         WAIT;
```

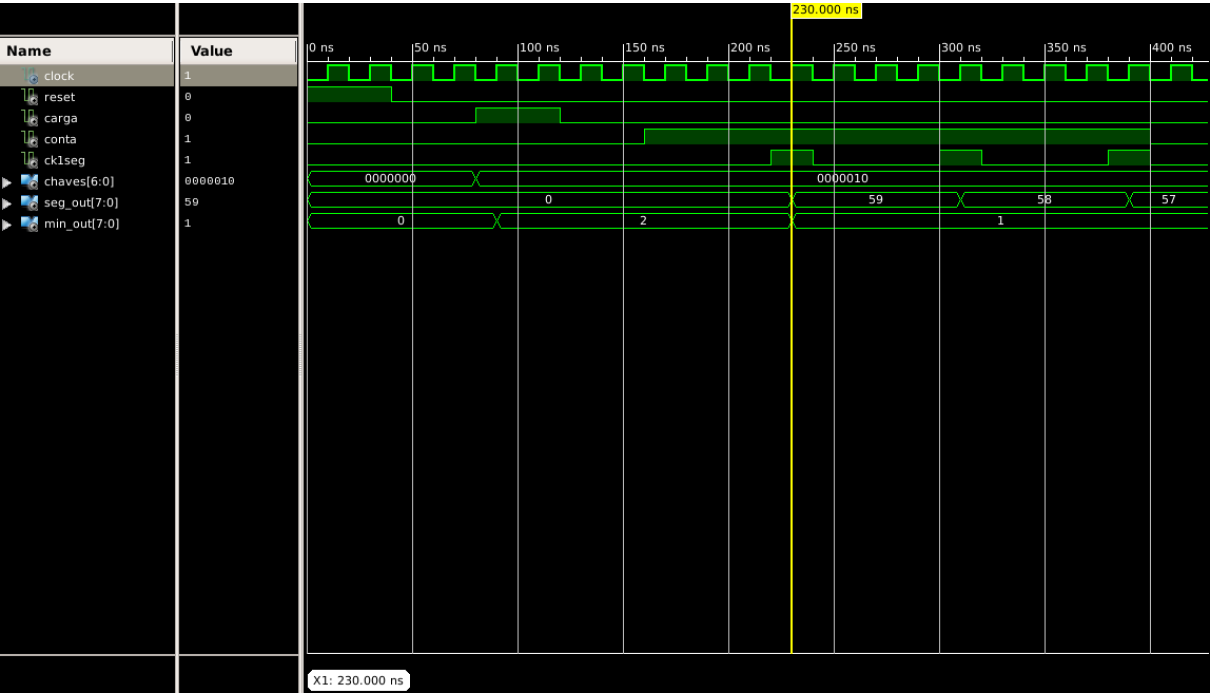
55 END PROCESS;

56 END behavior;

Fonte: Do autor (2026)

No processo de estímulos, o sistema é inicialmente resetado. Logo em seguida, o sinal carga é ativado enquanto o vetor `chaves` recebe o valor binário correspondente a 2, configurando o cronômetro para iniciar em "02:00". Após desativar a carga, o sinal de contagem é habilitado. Para simular a passagem do tempo de forma eficiente sem a necessidade de instanciar o divisor de clock real, foi utilizado um laço de repetição (`FOR LOOP`). Esse laço injeta pulsos rápidos na entrada `ck1seg`, permitindo observar o decréscimo imediato do tempo. A Figura 3 ilustra o resultado da simulação, destacando o momento exato em que os segundos, ao receberem o primeiro pulso, realizam a virada de 00 para 59, enquanto os minutos decaem corretamente de 2 para 1.

Figura 3 – Forma de ondas do contador decrescente.



Fonte: Do autor (2026).

2.1.3 Máquina de estados finitos

A FSM é o módulo responsável por coordenar o que irá acontecer com base nas ações do usuário. Para a Implementação do cronômetro, foram definidos três estados de operação, `IDLE`, em que a máquina ficará parada esperando o usuário interagir, `LOAD`, em que o usuário irá definir os minutos que serão cronometrados, e `COUNT`, sendo a função de cronometrar em si. A `ARCHITECTURE` foi desenvolvida utilizando dois processos. O primeiro, tendo como lista de sensibilidade o clock e o reset, é responsável por memorizar o estado atual a cada subida de

clock e processar o sinal de reset assíncrono. O segundo processo avalia as entradas do circuito e o estado atual para determinar o próximo estado lógico e as saídas de controle `carga_out` e `conta_out` que comandarão os contadores.

2.1.3.1 Implementação

Segue abaixo o código de Implementação da Máquina de Estados Finitos:

fsm_cron.vhd

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY fsm_cron IS
5      PORT(
6          clock, reset, carga_btn, conta_btn, fim_cont: IN std_logic;
7          carga_out, conta_out, led_parado: OUT std_logic
8      );
9  END ENTITY fsm_cron;
10
11 ARCHITECTURE behavior OF fsm_cron IS
12     TYPE states IS (IDLE, LOAD, COUNT);
13     SIGNAL estado_atual, proximo_estado: states;
14 BEGIN
15     PROCESS (clock, reset)
16     BEGIN
17         IF reset = '1' THEN
18             estado_atual <= IDLE;
19             led_parado <= '0';
20         ELSIF rising_edge(clock) THEN
21             estado_atual <= proximo_estado;
22             IF proximo_estado = LOAD THEN
23                 led_parado <= '0';
24             ELSIF estado_atual = COUNT AND proximo_estado = IDLE THEN
25                 led_parado <= '1';
26             END IF;
27         END IF;
28     END PROCESS;
29
30     PROCESS (estado_atual, carga_btn, conta_btn, fim_cont)
31     BEGIN
32         proximo_estado <= estado_atual;
33         carga_out <= '0';
34         conta_out <= '0';
35         CASE estado_atual IS
36             WHEN IDLE =>
37                 IF carga_btn = '1' THEN
38                     proximo_estado <= LOAD;

```

```

39         END IF;
40     WHEN LOAD =>
41         carga_out <= '1';
42         IF conta_btn = '1' THEN
43             proximo_estado <= COUNT;
44         END IF;
45     WHEN COUNT =>
46         conta_out <= '1';
47         IF fim_cont = '1' THEN
48             proximo_estado <= IDLE;
49         END IF;
50     END CASE;
51 END PROCESS;
52 END behavior;

```

Fonte: Do autor (2026)

No processo combinacional, a lógica de transição foi implementada através de uma estrutura CASE. No estado inicial IDLE, o sistema aguarda a detecção de nível alto no sinal `carga_btn` para transitar ao estado LOAD. Neste novo estado, o sinal de saída `carga_out` é ativado em '1', permitindo que o módulo contador registre o valor das chaves. O sistema permanece em LOAD até que o sinal `conta_btn` seja acionado, migrando para o estado final COUNT, onde a saída `conta_out` é habilitada para dar início ao decréscimo do tempo. A máquina retorna ao estado IDLE somente por um reset externo ou quando recebe a flag `fim_cont`, vinda do contador, indicando que o tempo atingiu zero.

Um detalhe importante da especificação do projeto é relacionado ao LED parado, que só acende quando o cronômetro passar para o estado IDLE pelo fim da contagem, e não quando ocorre o acionamento do reset.

2.1.3.2 Testbench

Para demonstração e validação da Máquina de Estados Finitos, foi desenvolvido o *testbench* apresentado abaixo:

fsm_cron_tb.vhd

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY tb_fsm IS
5  END ENTITY tb_fsm;
6
7  ARCHITECTURE behavior OF tb_fsm IS
8      SIGNAL clock: std_logic := '0';
9      SIGNAL reset: std_logic := '0';
10     SIGNAL carga_btn: std_logic := '0';

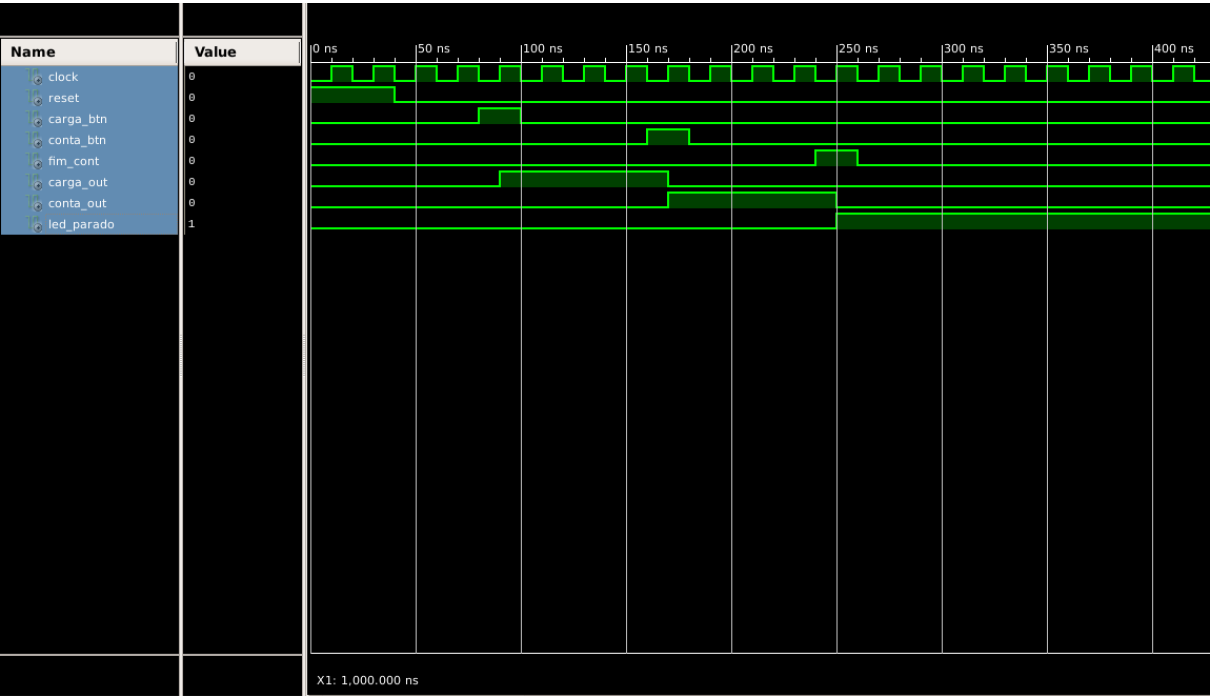
```

```
11     SIGNAL conta_btn: std_logic := '0';
12     SIGNAL fim_cont: std_logic := '0';
13     SIGNAL carga_out: std_logic;
14     SIGNAL conta_out: std_logic;
15     SIGNAL led_parado: std_logic;
16 BEGIN
17     UUT: ENTITY work.fsm_cron
18         PORT MAP(
19             clock => clock,
20             reset => reset,
21             carga_btn => carga_btn,
22             conta_btn => conta_btn,
23             fim_cont => fim_cont,
24             carga_out => carga_out,
25             conta_out => conta_out,
26             led_parado => led_parado
27         );
28     clock_process: PROCESS
29 BEGIN
30         clock <= '0';
31         WAIT FOR 10 ns;
32         clock <= '1';
33         WAIT FOR 10 ns;
34     END PROCESS;
35     fsm_process: PROCESS
36 BEGIN
37         reset <= '1';
38         WAIT FOR 40 ns;
39         reset <= '0';
40         WAIT FOR 40 ns;
41         carga_btn <= '1';
42         WAIT FOR 20 ns;
43         carga_btn <= '0';
44         WAIT FOR 60 ns;
45         conta_btn <= '1';
46         WAIT FOR 20 ns;
47         conta_btn <= '0';
48         WAIT FOR 60 ns;
49         fim_cont <= '1';
50         WAIT FOR 20 ns;
51         fim_cont <= '0';
52         WAIT FOR 60 ns;
53         WAIT;
54     END PROCESS;
55 END behavior;
```


No processo de estímulos, foi recriado o comportamento esperado de um usuário interagindo com os botões físicos da placa. Primeiramente, aplica-se o sinal de reinicialização. Em seguida, gera-se um pulso em `carga_btn` seguido de uma espera para verificar a estabilidade do estado lógico de carga. Logo após, aplica-se um pulso em `conta_btn`, levando o sistema ao estado de contagem, o qual emite o sinal de saída constante que será roteado aos contadores.

Para finalizar a simulação, é enviado um pulso no sinal `fim_cont`. Observando a forma de onda gerada apresentada abaixo, é possível acompanhar facilmente as mudanças de estado da máquina. Fica claro que, no exato momento em que a FSM recebe o aviso de que o tempo acabou e volta para o estado inicial `IDLE`, o sinal `led_parado` acende, funcionando exatamente como o esperado pelas regras do projeto.

Figura 4 – Forma de ondas da Máquina de Estados Finitos.



Fonte: Do autor (2026).

2.2 INTEGRAÇÃO DOS MÓDULOS

Após o desenvolvimento e validação de todos os módulos de forma individual, a etapa final do projeto consiste na integração de todas as partes. O arquivo `top.vhd` atua como o topo da hierarquia, funcionando como a "placa mãe" do projeto. O objetivo deste módulo não é criar novas lógicas sequenciais, mas sim instanciar os componentes já criados e conectá-los através de sinais internos.

2.2.1 Implementação do cronômetro decrescente

O código do arquivo de topo foi estruturado de forma a mapear os pinos físicos da placa para os componentes internos e rotear as informações entre eles.

top.vhd

```

1  LIBRARY ieee;
2  USE ieee.STD_LOGIC_1164.ALL;
3
4  ENTITY cron_dec IS
5      GENERIC(CLOCK_FREQ: integer := 50_000_000);
6      PORT(
7          clock, reset, carga, conta: IN std_logic;
8          chaves: IN std_logic_vector(6 DOWNTO 0);
9          parado: OUT std_logic;
10         an: OUT std_logic_vector(3 DOWNTO 0);
11         dec_ddp: OUT std_logic_vector(7 DOWNTO 0)
12     );
13 END ENTITY cron_dec;
14
15 ARCHITECTURE cron_dec OF cron_dec IS
16     type ROM is array (0 to 99) of std_logic_vector (7 downto 0);
17     constant conv_to_BCD : ROM:=
18         "00000000", "00000001", "00000010", "00000011", "00000100",
19         "00000101", "00000110", "00000111", "00001000", "00001001",
20         "00010000", "00010001", "00010010", "00010011", "00010100",
21         "00010101", "00010110", "00010111", "00011000", "00011001",
22         "00100000", "00100001", "00100010", "00100011", "00100100",
23         "00100101", "00100110", "00100111", "00101000", "00101001",
24         "00110000", "00110001", "00110010", "00110011", "00110100",
25         "00110101", "00110110", "00110111", "00111000", "00111001",
26         "01000000", "01000001", "01000010", "01000011", "01000100",
27         "01000101", "01000110", "01000111", "01001000", "01001001",
28         "01010000", "01010001", "01010010", "01010011", "01010100",
29         "01010101", "01010110", "01010111", "01011000", "01011001",
30         "01100000", "01100001", "01100010", "01100011", "01100100",
31         "01100101", "01100110", "01100111", "01101000", "01101001",
32         "01110000", "01110001", "01110010", "01110011", "01110100",
33         "01110101", "01110110", "01110111", "01111000", "01111001",
34         "10000000", "10000001", "10000010", "10000011", "10000100",
35         "10000101", "10000110", "10000111", "10001000", "10001001",
36         "10010000", "10010001", "10010010", "10010011", "10010100",
37         "10010101", "10010110", "10010111", "10011000", "10011001");
38
39     TYPE states IS (IDLE, LOAD, COUNT);
40     SIGNAL NST, PST: states;
41     SIGNAL ck1seg, carga_out, conta_out, fim_cont: std_logic;
42     SIGNAL seg, min, Segundos_BCD, Minutos_BCD: std_logic_vector(7
43         DOWNTO 0);
44     SIGNAL d4, d3, d2, d1: std_logic_vector(5 DOWNTO 0);
45 BEGIN

```

```

46 U1: ENTITY work.divisor_clock
47     GENERIC MAP(
48         CLOCK_FREQ => CLOCK_FREQ
49     )
50     PORT MAP(
51         clock => clock,
52         reset => reset,
53         ck1seg => ck1seg
54     );
55
56 U2: ENTITY work.fsm_cron
57     PORT MAP(
58         clock => clock,
59         reset => reset,
60         carga_btn => carga,
61         conta_btn => conta,
62         fim_cont => fim_cont,
63         carga_out => carga_out,
64         conta_out => conta_out,
65         led_parado => parado
66     );
67
68 U3: ENTITY work.contador_dec
69     PORT MAP(
70         clock => clock,
71         reset => reset,
72         chaves => chaves,
73         carga => carga_out,
74         conta => conta_out,
75         ck1seg => ck1seg,
76         seg_out => seg,
77         min_out => min
78     );
79
80 fim_cont <= '1' WHEN (seg = "00000000" AND min = "00000000") ELSE
81     '0';
82
83 Segundos_BCD <= conv_to_BCD(to_integer(unsigned(seg)));
84 Minutos_BCD <= conv_to_BCD(to_integer(unsigned(min)));
85
86 d1 <= '1' & Segundos_BCD(3 DOWNTO 0) & '1';
87 d2 <= '1' & Segundos_BCD(7 DOWNTO 4) & '1';
88 d3 <= '1' & Minutos_BCD(3 DOWNTO 0) & '0';
89 d4 <= '1' & Minutos_BCD(7 DOWNTO 4) & '1';
90
91 display_driver: ENTITY WORK.dspl_drv
92     PORT MAP(

```

```

92         clock => clock ,
93         reset => reset ,
94         d3 => d4 ,
95         d2 => d3 ,
96         d1 => d2 ,
97         d0 => d1 ,
98         an => an ,
99         dec_ddp => dec_ddp
100     );
101 END cron_dec ;

```

Fonte: Do autor (2026)

Dentro da entidade, foi criada a variável `GENERIC` chamada `CLOCK_FREQ` com o valor de 50000000, para repassar essa informação ao divisor de clock. No `PORT`, foram definidas as entradas físicas que vêm da placa (clock, reset, carga, conta e as chaves), e as saídas que irão para o mostrador e para o LED de parada (parado, an, e dec_ddp).

Na `ARCHITECTURE`, foi passado pelo professor e implementado um tipo ROM e uma constante chamada `conv_to_BCD`. Como o professor já forneceu a lógica do *display*, essa matriz foi utilizada para traduzir diretamente os valores de 0 a 99 em binário puro para o formato BCD (Decimal Codificado em Binário). Abaixo dela, também foram definidos os sinais (`SIGNAL`) que farão o papel de "fios" para conectar as portas de um módulo ao outro.

Logo após o `BEGIN`, foram instanciados os três módulos principais desenvolvidos: o divisor de clock (U1), a máquina de estados (U2) e o contador decrescente (U3), mapeando as portas deles com os sinais internos.

Para a máquina de estados funcionar corretamente, foi definida uma condição no sinal `fim_cont`, em que ele recebe '1' apenas quando os segundos e os minutos do contador chegam a "00000000" ao mesmo tempo. Esse sinal é enviado para a FSM saber que o tempo acabou e poder acender o LED de parada.

Para a exibição no *display*, os sinais `seg` e `min` do contador são convertidos para inteiro e o valor correspondente é buscado na matriz ROM, sendo guardado em `Segundos_BCD` e `Minutos_BCD`. Em seguida, esses valores são concatenados para formar os dígitos `d1`, `d2`, `d3` e `d4`. No caso do `d3`, foi colocado um '0' na última posição propositalmente para acender o ponto do *display* da placa, separando visualmente os minutos dos segundos.

Por fim, foi instanciado o módulo `dsp1_drv` (fornecido pelo professor), passando esses dígitos formatados para que ele faça a multiplexação e exiba os valores corretamente no mostrador da Nexys 1.

2.2.2 Testbench

Para a validação final do cronômetro decrescente, foi utilizado o *testbench* fornecido pelo professor, responsável por simular o funcionamento global do cronômetro.

top_tb.vhd

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY tb_fsm IS
5  END ENTITY tb_fsm;
6
7  ARCHITECTURE behavior OF tb_fsm IS
8      SIGNAL clock: std_logic := '0';
9      SIGNAL reset: std_logic := '0';
10     SIGNAL carga_btn: std_logic := '0';
11     SIGNAL conta_btn: std_logic := '0';
12     SIGNAL fim_cont: std_logic := '0';
13     SIGNAL carga_out: std_logic;
14     SIGNAL conta_out: std_logic;
15     SIGNAL led_parado: std_logic;
16 BEGIN
17     UUT: ENTITY work.fsm_cron
18         PORT MAP(
19             clock => clock,
20             reset => reset,
21             carga_btn => carga_btn,
22             conta_btn => conta_btn,
23             fim_cont => fim_cont,
24             carga_out => carga_out,
25             conta_out => conta_out,
26             led_parado => led_parado
27         );
28     clock_process: PROCESS
29 BEGIN
30         clock <= '0';
31         WAIT FOR 10 ns;
32         clock <= '1';
33         WAIT FOR 10 ns;
34     END PROCESS;
35     fsm_process: PROCESS
36 BEGIN
37         reset <= '1';
38         WAIT FOR 40 ns;
39         reset <= '0';
40         WAIT FOR 40 ns;
41         carga_btn <= '1';
42         WAIT FOR 20 ns;
43         carga_btn <= '0';
44         WAIT FOR 60 ns;
45         conta_btn <= '1';
46         WAIT FOR 20 ns;
```

```
47     conta_btn <= '0';
48     WAIT FOR 60 ns;
49     fim_cont <= '1';
50     WAIT FOR 20 ns;
51     fim_cont <= '0';
52     WAIT FOR 60 ns;
53     WAIT;
54 END PROCESS;
55 END behavior;
```

Fonte: Do autor (2026)

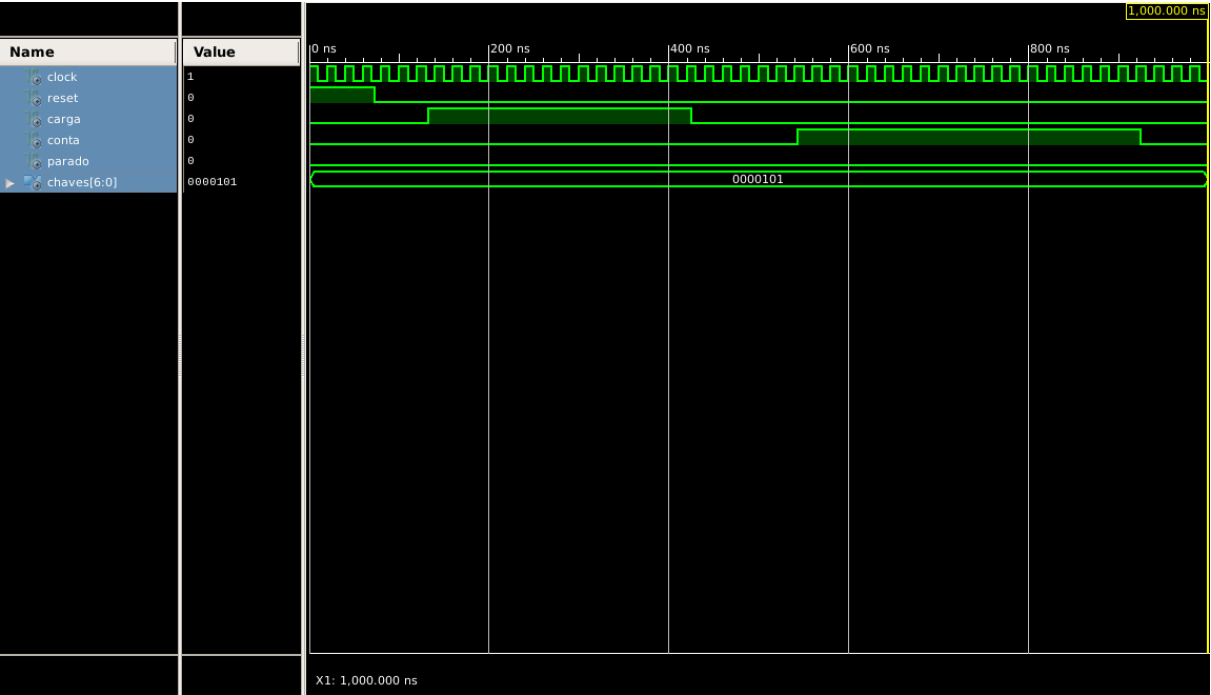
Na ARCHITECTURE, os sinais correspondentes às entradas e saídas do sistema foram definidos. Logo após o BEGIN, o funcionamento do cristal da placa foi simulado alternando o valor lógico do clock a cada 10 nanosegundos.

Para os sinais de controle, os estímulos foram programados em uma única linha de tempo. O reset inicia ativado e é desligado após 73 ns. O sinal de carga é ativado em 133 ns para que o valor fixo de 5 minutos, definido na variável *chaves* ("0000101"), seja gravado no contador. Posteriormente, a carga é desativada e o sinal de conta entra em nível alto em 543 ns, dando início efetivo à contagem decrescente, sendo desativado em 925 ns para simular a pausa do cronômetro.

Na instanciação do módulo de topo (UUT), a variável genérica *CLOCK_FREQ* foi novamente reescrita para o valor 4. Como explicado nos módulos anteriores, essa técnica foi aplicada para evitar que o simulador precise processar 50 milhões de ciclos de clock para avançar apenas 1 segundo no tempo, viabilizando a visualização das formas de onda.

Um detalhe importante na etapa de mapeamento das portas (PORT MAP) é o uso da palavra-chave *open* nas saídas *an* e *dec_ddp*. Como essas saídas representam os pinos físicos do mostrador multiplexado, a sua leitura visual na forma de onda não são úteis para a validação do cronômetro. Portanto, essas portas foram deixadas intencionalmente abertas durante a simulação.

Figura 5 – Forma de ondas do cronômetro decrescente.



Fonte: Do autor (2026).

A imagem acima apresenta o resultado da simulação do arquivo de topo. Nela, é possível observar o comportamento dos sinais de controle operando exatamente conforme programado na linha de estímulos do testbench.

2.3 PROTOTIPAÇÃO NA PLACA NEXYS 1

Para que o código desenvolvido e simulado em ambiente virtual pudesse ser testado fisicamente, foi realizada a prototipação do sistema na placa FPGA Nexys 1. O passo mais importante para essa integração física é a criação do arquivo de restrições do usuário, conhecido como UCF (User Constraint File). O objetivo deste arquivo é mapear as portas de entrada e saída, que foram definidas na entidade de topo do projeto, para os pinos físicos exatos onde os componentes estão soldados na placa.

2.3.1 Implementação do mapeamento

Segue abaixo o código de mapeamento dos pinos no arquivo UCF.

pinos.ucf

```
1 NET "clock" LOC = "T9";
2
3 NET "reset" LOC = "J13";
4 NET "conta" LOC = "K13";
5 NET "carga" LOC = "K12";
6
```

```

7 NET "chaves[6]" LOC = "M15";
8 NET "chaves[5]" LOC = "M16";
9 NET "chaves[4]" LOC = "L15";
10 NET "chaves[3]" LOC = "K15";
11 NET "chaves[2]" LOC = "K16";
12 NET "chaves[1]" LOC = "J16";
13 NET "chaves[0]" LOC = "N15";
14
15 NET "parado" LOC = "L14";
16
17 NET "an[3]" LOC = "F12";
18 NET "an[2]" LOC = "G13";
19 NET "an[1]" LOC = "G12";
20 NET "an[0]" LOC = "G14";
21
22 NET "dec_ddp[7]" LOC = "F13";
23 NET "dec_ddp[6]" LOC = "E13";
24 NET "dec_ddp[5]" LOC = "G15";
25 NET "dec_ddp[4]" LOC = "H13";
26 NET "dec_ddp[3]" LOC = "J14";
27 NET "dec_ddp[2]" LOC = "E14";
28 NET "dec_ddp[1]" LOC = "G16";
29 NET "dec_ddp[0]" LOC = "H14";

```

Fonte: Do autor (2026)

Como pode ser observado no arquivo de restrições, o sinal de clock foi mapeado para o pino T9, que corresponde ao oscilador de cristal de 50 MHz da placa. Os sinais de controle do sistema (reset, conta e carga) foram associados a três botões de pressão (*push buttons*). As sete chaves deslizantes (*switches*) foram conectadas ao vetor *chaves* para permitir a inserção manual dos minutos desejados.

O LED de indicação de fim de contagem, representado pelo sinal *parado*, foi roteado para o pino L14. Por fim, os sinais de controle do *display* multiplexado (*an* para os ânodos e *dec_ddp* para os cátodos e o ponto decimal) foram mapeados para os pinos do mostrador de sete segmentos da Nexys 1.

2.3.2 Validação prática

Após a configuração do UCF, o projeto foi sintetizado e o arquivo binário gerado (bitsream) foi transferido para a FPGA. Durante os testes práticos, o sistema comportou-se exatamente como o previsto nas simulações das formas de onda. A interação com a máquina de estados através dos botões, a carga correta do tempo escolhido pelas chaves e o decréscimo visual no mostrador síncrono a cada segundo foram validados com sucesso.

Para demonstrar o funcionamento real e a estabilidade do cronômetro operando na placa

física, foi gravado um vídeo demonstrativo, que pode ser acessado através do link a seguir:
<https://youtu.be/rcNc9c0OMGk>

3 CONCLUSÃO

Com este trabalho, foi possível concluir com sucesso a criação, simulação e prototipação de um cronômetro decrescente utilizando a linguagem VHDL. A escolha de usar um design hierárquico, dividindo o código em módulos menores, facilitou bastante o desenvolvimento e os testes individuais de cada parte do circuito.

A construção de cada módulo reforçou como o hardware funciona na prática. O divisor de clock garantiu que os 50 MHz da placa virassem pulsos exatos de 1 segundo para o tempo não passar rápido demais. A decisão de juntar os contadores de minutos e segundos em um só bloco tornou o roteamento mais simples e facilitou a matemática da contagem. Tudo isso foi muito bem organizado e coordenado pela máquina de estados (FSM).

A etapa de simulação com os testbenches foi essencial para comprovar que a lógica estava certa antes de ir para o mundo real, permitindo ver as formas de onda de cada módulo funcionando perfeitamente. Por fim, a prototipação na placa Nexys 1 foi a prova final do projeto. O mapeamento dos pinos pelo arquivo UCF funcionou, e os componentes físicos da placa (botões, chaves e o mostrador de 7 segmentos) responderam exatamente como projetado e simulado. Isso confirmou que o circuito modelado funcionou perfeitamente na prática.